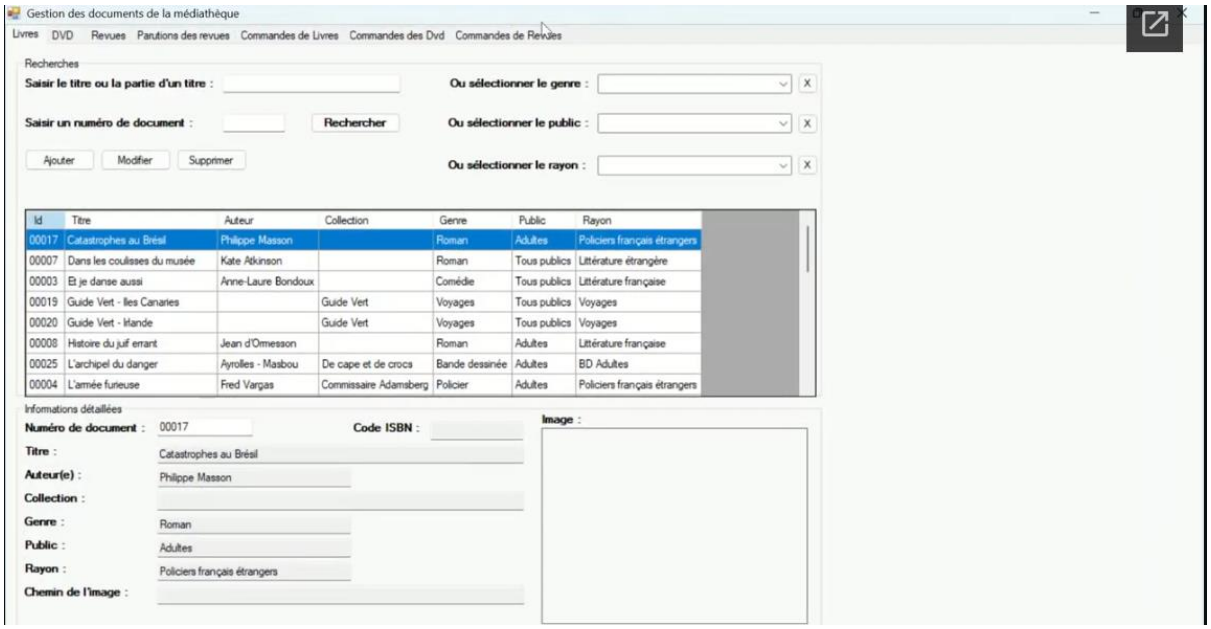
AP2 – MediaTekDocuments



Sarita Nasimiyu

Contexte général 2
Contexte 2
Existant 3
Demande 3
Langages et technologies utilisés 3
Mission 1 – Gestion des documents 3
Demande de la tâche 4
Travail réalisé 4
Étapes suivies 5
Résultat obtenu 6
Mission 2 – Gérer les commandes 7
Tâche 1 – Gérer les commandes de livres et de DVD 7
Demande de la tâche 7
Travail réalisé 7
Étapes suivies 9
Résultat obtenu 10
Tâche 2 – Gérer les commandes de revues 11
Demande de la tâche 11
Étapes suivies 12
Résultat obtenu 13
Mission 4 – Gérer les authentifications et les droits d'accès 13
Demande de la tâche 13
Travail réalisé 13
Étapes suivies 14
Résultat obtenu 15
Mission 5 – Assurer la sécurité, la qualité et intégrer des logs 15
Tâche 1 – Corriger des problèmes de sécurité 16
Demande de la tâche 16
Travail réalisé 16
Étapes suivies 16

Résultat obtenu 16
Tâche 2 – Contrôler la qualité 18
Demande de la tâche 18
Travail réalisé 18
Étapes suivies 18
Résultat obtenu 18
Tâche 3 – Intégrer des logs 19
Demande de la tâche 19
Travail réalisé 19
Étapes suivies 19
Résultat obtenu 20
Mission 6 – Tester et documenter 20
Tâche 1 – Gérer les tests 20
Demande de la tâche 21
Travail réalisé 21
Étapes suivies 21
Résultat obtenu 21
Tâche 2 – Créer les documentations techniques 22
Demande de la tâche 22
Travail réalisé 22
Étapes suivies 23
Résultat obtenu 23
Tâche 3 – Créer la documentation utilisateur en vidéo 24
Demande de la tâche 24
Travail réalisé 24
Étapes suivies 24
Résultat obtenu 25
Conclusion générale 25

Contexte général

Contexte

L'atelier AP2 - MediaTekDocuments s'inscrit dans un contexte de reprise et d'évolution d'une application de bureau existante. Cette application a pour objectif de permettre la gestion des documents d'une médiathèque (livres, DVD et revues), ainsi que le suivi des commandes et des abonnements, en s'appuyant sur une API REST connectée à une base de données MySQL.

Le travail demandé dans le cadre de cet atelier consistait à intervenir sur une base applicative déjà fournie, à la faire évoluer en ajoutant de nouvelles fonctionnalités (gestion des commandes, authentification, sécurité), à corriger certains points techniques, à améliorer la

qualité du code, à mettre en place des tests, à intégrer des logs, puis à documenter l'ensemble des fonctionnalités développées.

Existant

Une application partielle existait déjà permettant la consultation des documents et des revues. Le projet reposait sur une architecture C# avec une base de données relationnelle MySQL et une API rest.

Demande

Le projet demandait d'ajouter les fonctionnalités suivantes :

- Gérer les documents
- Gérer les commandes
- Mettre en place des authentifications
- Assurer la sécurité, la qualité et intégrer des logs
- Tester et documenter

Langages et technologies utilisés

Les principaux langages et technologies utilisés dans le cadre de cet atelier sont les suivants :

- **C#** : développement de l'application de bureau MediaTekDocuments ;
- **MySQL** : gestion de la base de données ;
- **API REST** : communication entre l'application C# et la base de données ;
- **Visual Studio 2022** : environnement de développement pour l'application C# ;
- **XAMPP** : environnement local pour héberger l'API et la base de données ;
- **Postman** : tests des requêtes API ;
- **GitHub** : gestion de version et suivi des corrections (issues, pull requests) ;
- **Serilog** : gestion des logs de l'application ;
- **SonarQube / SonarLint** : analyse de la qualité du code ;
- **phpDocumentor / documentation XML** : génération de la documentation technique;

Mission 1 – Gestion des documents

Demande de la tâche

Cette tâche consistait à ajouter, dans les onglets Livres, DVD et Revues de l'application MediaTekDocuments, les fonctionnalités permettant de créer, modifier et supprimer un document.

Temps estimé : 8 heures
Temps réel : 11 heures

Travail réalisé

Le travail a consisté à compléter l'application afin de permettre une gestion complète des documents dans les trois onglets concernés.

Les principales fonctionnalités ajoutées sont :

- L'ajout d'un document (livre, DVD ou revue);
- La modification d'un document existant ;
- La suppression d'un document sous conditions (absence d'exemplaires et de commandes).

Des évolutions importantes ont également été apportées à l'interface afin d'améliorer l'expérience utilisateur et limiter les erreurs :

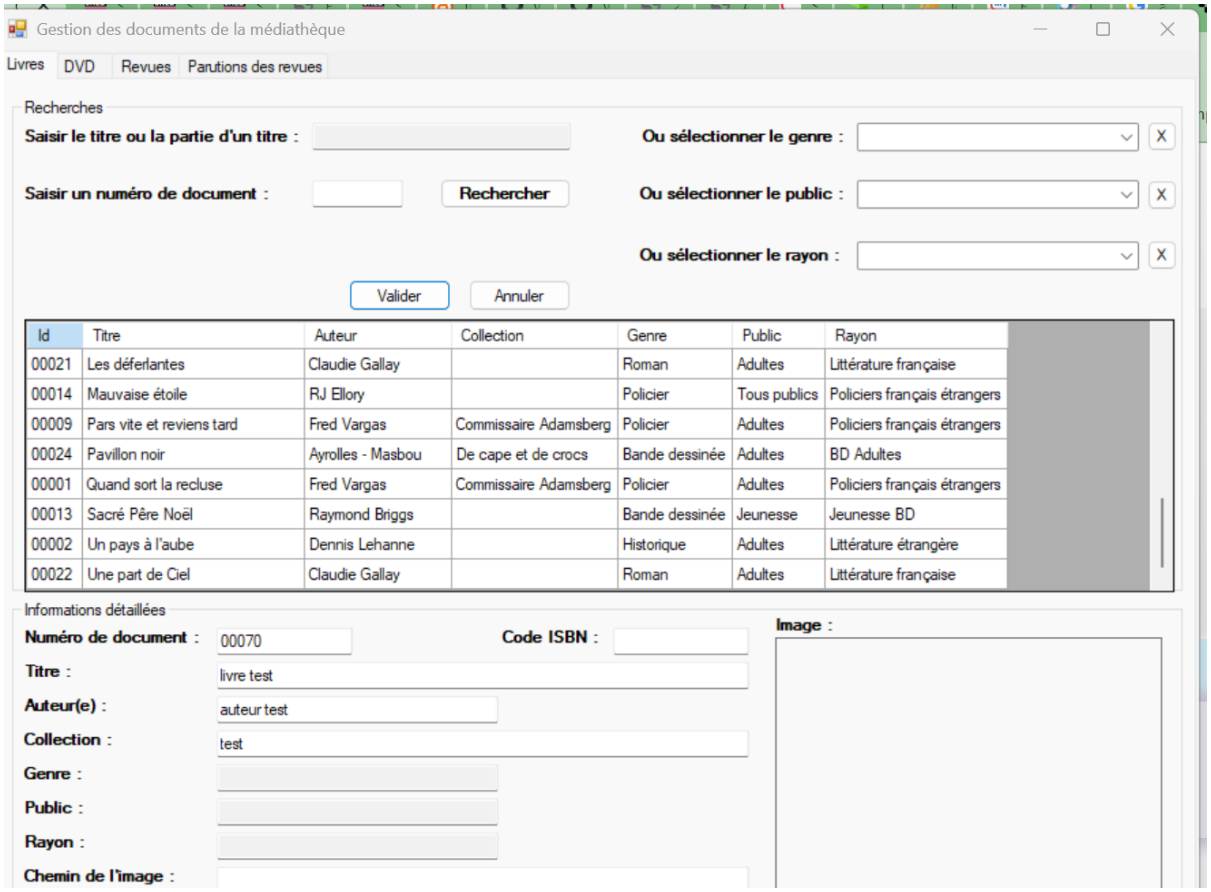
- Mise en place d'un mode édition avec affichage des boutons Valider / Annuler ;
- Désactivation des champs en mode consultation ;
- Ajout de boutons / méthodes pour l'ajout, la modification et la suppression des documents

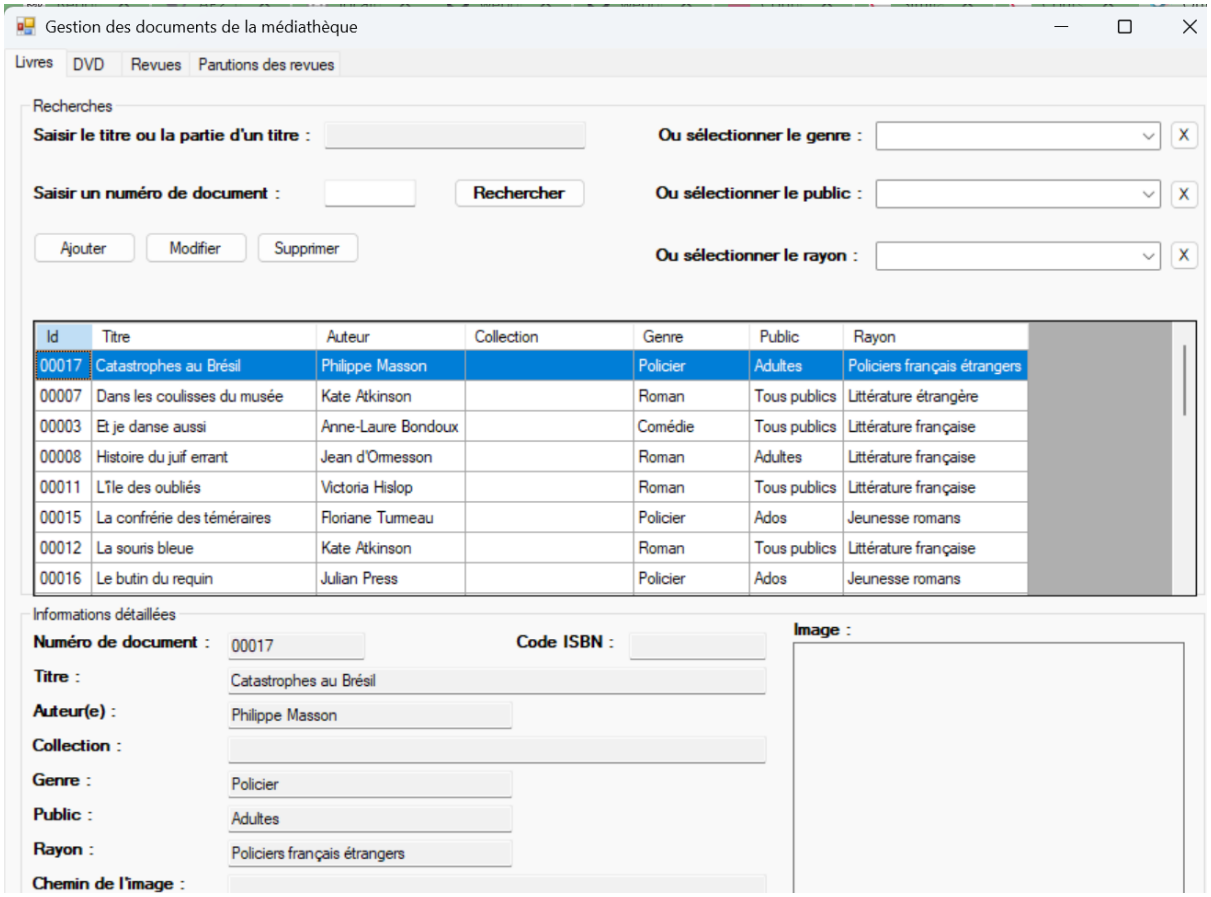
```
188     /// <summary>
189     /// Met les champs en mode édition ou lecture seule
190     /// </summary>
191     /// <param name="modeEdition">true = édition, false = lecture</param>
        5 references | webby!, 18 days ago | 1 author, 3 changes
192     private void ModeEditionLivres(bool modeEdition)
193     {
194         txbLivresTitre.ReadOnly = !modeEdition;
195         txbLivresAuteur.ReadOnly = !modeEdition;
196         txbLivresCollection.ReadOnly = !modeEdition;
197         txbLivresIsbn.ReadOnly = !modeEdition;
198         txbLivresImage.ReadOnly = !modeEdition;
199         btnValiderLivre.Visible = modeEdition;
200         btnAnnulerLivre.Visible = modeEdition;
201         btnAjoutLivre.Visible = !modeEdition;
202         btnModifierLivre.Visible = !modeEdition;
203         btnSupprimerLivre.Visible = !modeEdition;
204         cbxLivresGenres.Enabled = !modeEdition;
205         cbxLivresPublics.Enabled = !modeEdition;
206         cbxLivresRayons.Enabled = !modeEdition;
207         txbLivresTitreRecherche.ReadOnly = !modeEdition;
208         txbLivresGenre.Visible = !modeEdition;
209         txbLivresPublic.Visible = !modeEdition;
210         txbLivresRayon.Visible = !modeEdition;
211         cbxLivresGenresEditAdd.Visible = modeEdition;
212         cbxLivresPublicsEditAdd.Visible = modeEdition;
213         cbxLivresRayonsEditAdd.Visible = modeEdition;
214
215         txbLivresNumRecherche.Enabled = !modeEdition;
216         txbLivresTitreRecherche.Enabled = !modeEdition;
217         btnLivresNumRecherche.Enabled = !modeEdition;
218         btnLivresAnnulGenres.Enabled = !modeEdition;
219         btnLivresAnnulRayons.Enabled = !modeEdition;
220     }
```

Étapes suivies

Le développement a été réalisé en plusieurs étapes :

- Mise en place des fonctionnalités CRUD pour les livres ;
- Adaptation de la logique pour les DVD et les revues ;
- Modification de l'interface graphique pour gérer les modes édition et consultation ;
- Implémentation des contrôles de saisie (champs obligatoires, types de données, unicité) ;
- Ajout de messages de confirmation et d'erreur ;
- Mise en place des contrôles de dépendance avant suppression.





Résultat obtenu

L'application permet désormais une gestion complète et sécurisée des documents.

Les règles métier sont respectées, notamment :

- Impossibilité de supprimer un document lié à des exemplaires ou des commandes ;
- Impossibilité de modifier l'identifiant d'un document existant ;
- Validation des données saisies avant enregistrement.

Le code reste structuré et cohérent avec l'architecture de l'application (Vue / Contrôleur / Accès aux données), et l'interface est plus claire et plus fiable pour l'utilisateur.

Mission 2 – Gérer les commandes

Tâche 1 – Gérer les commandes de livres et de DVD

Demande de la tâche

Cette tâche consistait à mettre en place la gestion des commandes pour les livres et les DVD dans l'application MediaTekDocuments.

L’application devait permettre :

- De créer une commande pour un ou plusieurs exemplaires ;
- De suivre l’évolution d’une commande selon différents états (en cours, relancée, livrée, réglée) ;
- D’afficher la liste des commandes associées à un document ;
- De supprimer une commande uniquement si elle n’est pas encore livrée ;
- De générer automatiquement les exemplaires lors du passage à l’état "livrée".

Temps estimé : 8 heures
Temps réel : 12 heures

Travail réalisé

Le travail a consisté à développer l’ensemble du module de gestion des commandes pour les livres et les DVD.

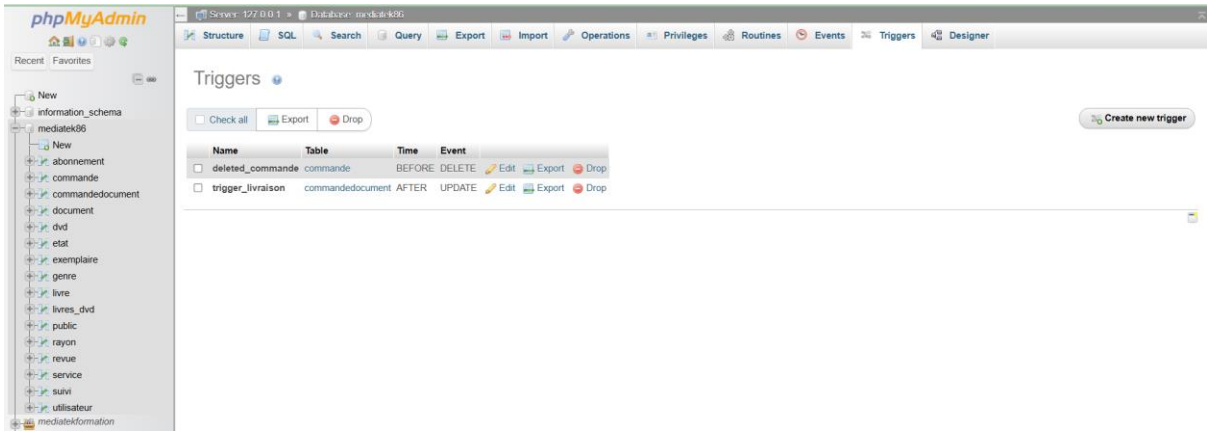
Les principales fonctionnalités mises en place sont :

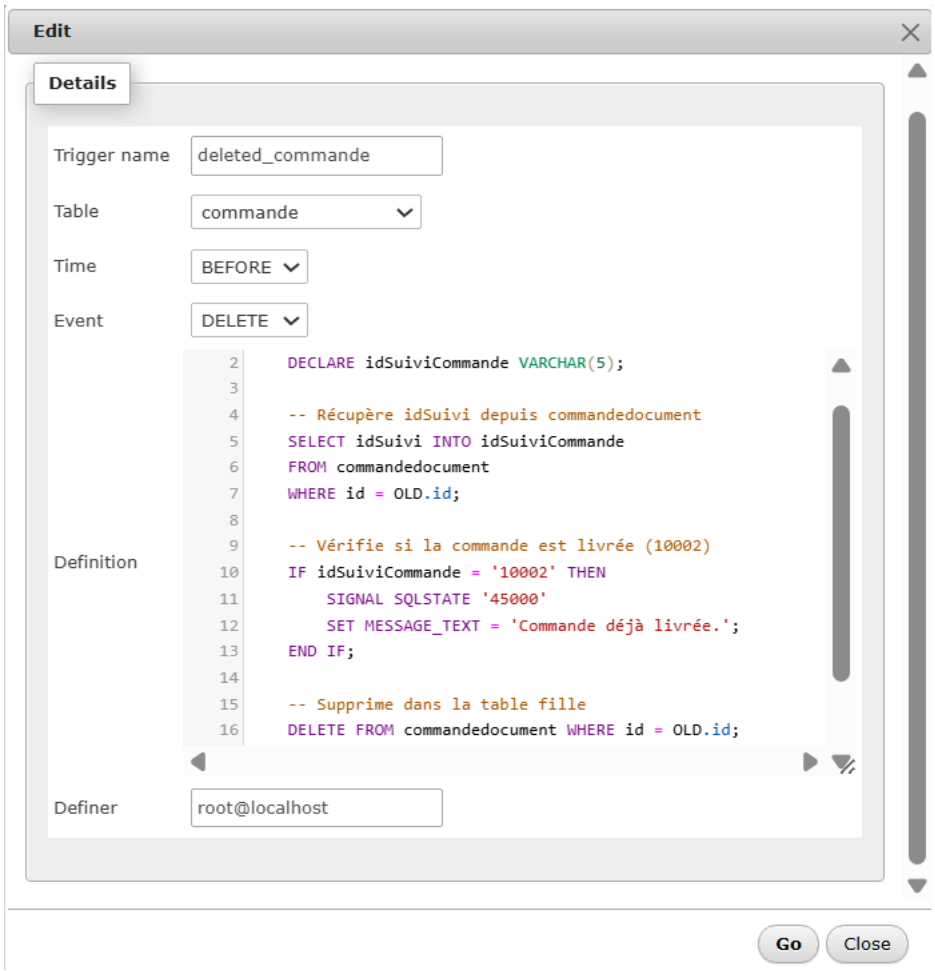
- La création de commandes avec saisie de la date, du montant et de la quantité ;
- L’affichage des commandes associées à un livre ou un DVD ;
- La gestion des différents états d’une commande (en cours, relancée, livrée, réglée) ;
- La modification de l’état d’une commande avec contrôle des transitions ;
- La suppression d’une commande uniquement si elle n’est pas livrée ;
- L’affichage du libellé des états à partir de la table Suivi.

Des règles métier ont également été intégrées :

- Une commande ne peut pas revenir à un état précédent ;
- Une commande ne peut pas être réglée si elle n’est pas livrée ;
- Une commande livrée ne peut plus être supprimée.

Des triggers ont été mis en place dans la base de données pour automatiser certaines règles métier, notamment la suppression cohérente d’une commande et des données associées.
Un second trigger permet de créer automatiquement les exemplaires lorsqu’une commande passe à l’état « livrée », avec une date d’achat issue de la commande, un état initial « neuf » et une numérotation séquentielle.



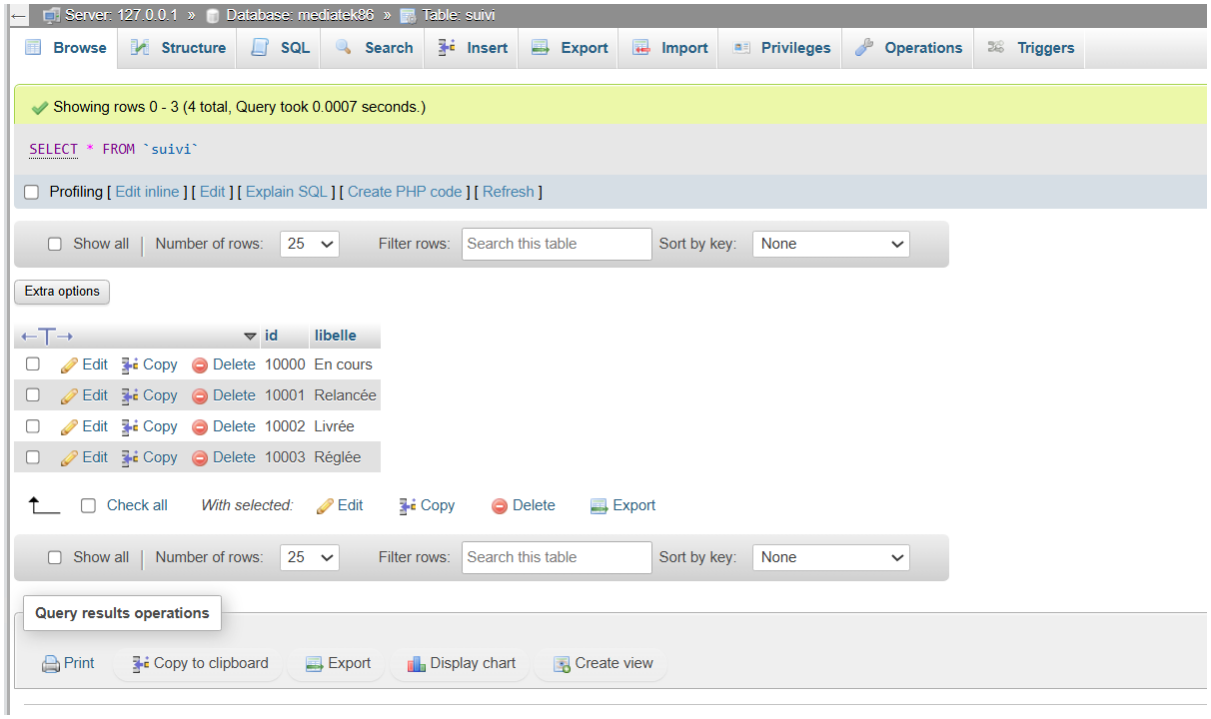


Étapes suivies

Le développement a été réalisé en plusieurs étapes :

- Création de la table Suivi et de la classe Suivi ;
- Récupération et affichage des commandes liées à un document ;
- Fusion des données provenant des tables Commande et CommandeDocument ;
- Mise en place du formulaire de création de commande ;
- Implémentation de la gestion des transitions d'état ;

- Ajout des contrôles empêchant les actions invalides (suppression ou retour en arrière);
- Mise à jour dynamique de l'interface après chaque action.



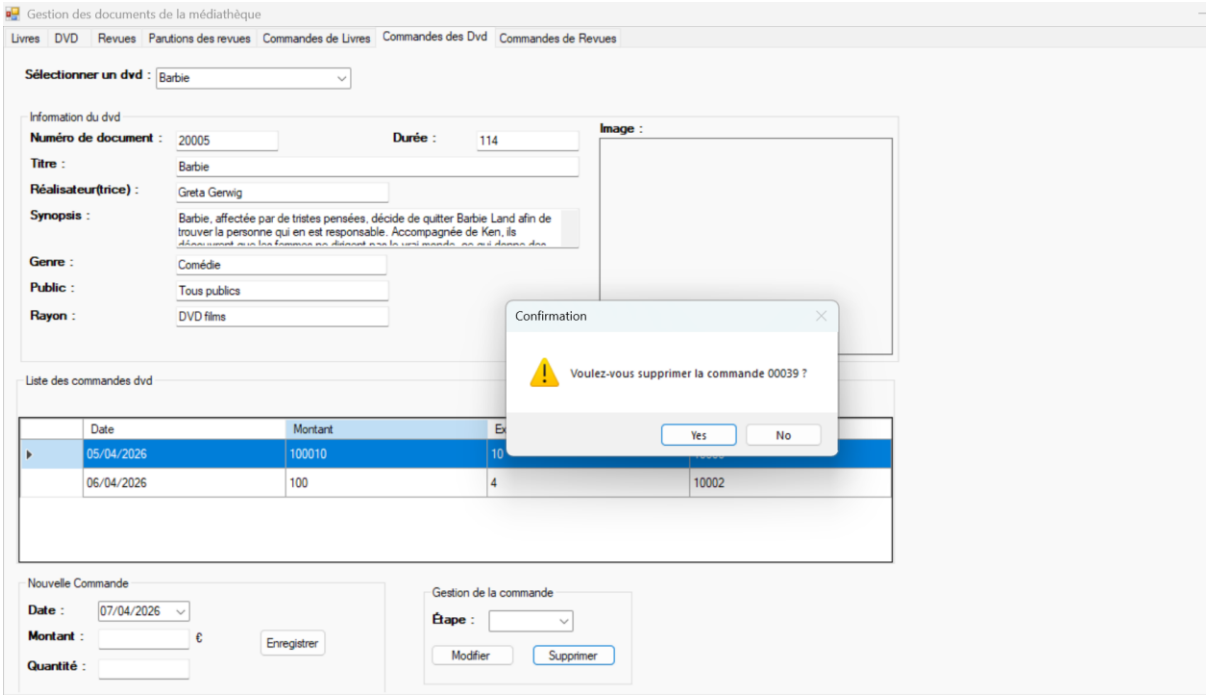
Résultat obtenu

Le module de gestion des commandes est désormais fonctionnel et permet un suivi complet du cycle de vie d'une commande.

Les contraintes sont respectées, notamment :

- Impossibilité de supprimer une commande livrée ;
- Impossibilité de revenir à un état précédent ;
- Respect de l'ordre logique des étapes (en cours → relancée → livrée → réglée).

L'interface permet une consultation claire des commandes et facilite leur gestion.
Le code reste structuré et cohérent avec l'architecture de l'application, garantissant sa maintenabilité et son évolutivité.



Tâche 2 – Gérer les commandes de revues

Demande de la tâche

Cette tâche consistait à mettre en place la gestion des commandes de revues dans l'application MediaTekDocuments.
Dans ce contexte, une commande de revue correspond à un abonnement ou à un renouvellement d'abonnement.
L'application devait permettre d'afficher la liste des abonnements, de supprimer un abonnement uniquement si aucun exemplaire n'y est rattaché, et d'afficher au démarrage une alerte pour les revues dont l'abonnement se termine dans moins de 30 jours.

Travail réalisé

Le travail a consisté à développer un module dédié à la gestion des abonnements de revues.

Les principales fonctionnalités mises en place sont :

- L'affichage de la liste des abonnements d'une revue;
- L'ajout d'un abonnement ou d'un renouvellement via un formulaire ;
- La suppression conditionnelle d'un abonnement ;

L'affichage automatique d'une fenêtre d'alerte au démarrage de l'application.

La suppression d'un abonnement a été sécurisée grâce à une vérification portant sur les exemplaires déjà enregistrés.
Pour cela, la méthode *ParutionDansAbonnement* a été utilisée afin de déterminer si une

parution appartient à la période couverte par l'abonnement.
Une méthode complémentaire a également été développée pour récupérer les abonnements se terminant dans moins de 30 jours et les afficher dans une fenêtre d'alerte triée par date croissante.

```
/// <summary>
/// Retourne les abonnements qui expirent dans moins de 30 jours
/// </summary>
1 reference | webby!, 14 days ago | 1 author, 1 change
public List<Abonnement> RecupererAbonnementsExpirants()
{

    List<Abonnement> abonnementsExpirants = new List<Abonnement>();
    // on recupere tous les abonnements
    List<Abonnement> tousAbonnements = controller.GetAllAbonnements();

    DateTime aujourdhui = DateTime.Today;
    DateTime dansUnMois = aujourdhui.AddDays(30);

    //on filtre ceux qui expirent dans < 30 jours
    foreach (Abonnement abo in tousAbonnements)
    {
        if (abo.DateFinAbonnement >= aujourdhui && abo.DateFinAbonnement < dansUnMois)
        {
            abonnementsExpirants.Add(abo);
        }
    }

    //on trie par date (ordre chronologique)


    abonnementsExpirants = abonnementsExpirants.OrderBy(a => a.DateFinAbonnement).ToList();

    return abonnementsExpirants;
}
```

```
/// <summary>
/// Retourne vrai si la date de parution de l'exemplaire est entre les 2 autres dates
/// </summary>

1 reference | webby!, 11 days ago | 1 author, 2 changes
private static bool ParutionDansAbonnement(DateTime dateCommande, DateTime dateFinAbonnement, DateTime dateParution)
{
    return dateParution >= dateCommande && dateParution <= dateFinAbonnement;
}
```

Étapes suivies

Le développement a été réalisé en plusieurs étapes :

- Création de l'onglet de gestion des commandes de revues ;
- Récupération et affichage des abonnements associés à une revue ;
- Mise en place d'ajout d'un abonnement ;
- Implémentation de la méthode `ParutionDansAbonnement` pour contrôler la suppression ;
- Ajout du contrôle empêchant la suppression si des exemplaires sont rattachés ;

- Développement de la méthode permettant d'identifier les abonnements expirant dans moins de 30 jours ;
- Affichage automatique de la fenêtre d'alerte au démarrage de l'application.

Résultat obtenu

La gestion des commandes de revues est désormais opérationnelle.
L'application permet de consulter les abonnements, de les ajouter, de les supprimer sous conditions et d'anticiper leur renouvellement grâce à la fenêtre d'alerte affichée au démarrage.

Mission 4 – Gérer les authentifications et les droits d'accès

Demande de la tâche

Cette mission avait pour objectif de mettre en place un système d'authentification dans l'application MediaTekDocuments afin de gérer les accès en fonction du profil des utilisateurs.

Temps estimé : 4 heures
Temps réel : 7 heures

Travail réalisé

Un système complet d'authentification et de gestion des droits a été développé.

Les principales fonctionnalités mises en place sont :

- La création d'un formulaire de connexion permettant la saisie du login et du mot de passe ;
- La vérification des identifiants via l'API ;
- La gestion des erreurs (champs vides, identifiants incorrects) ;
- Le blocage de l'accès pour certains profils non autorisés ;
- L'adaptation dynamique de l'interface en fonction du service de l'utilisateur.

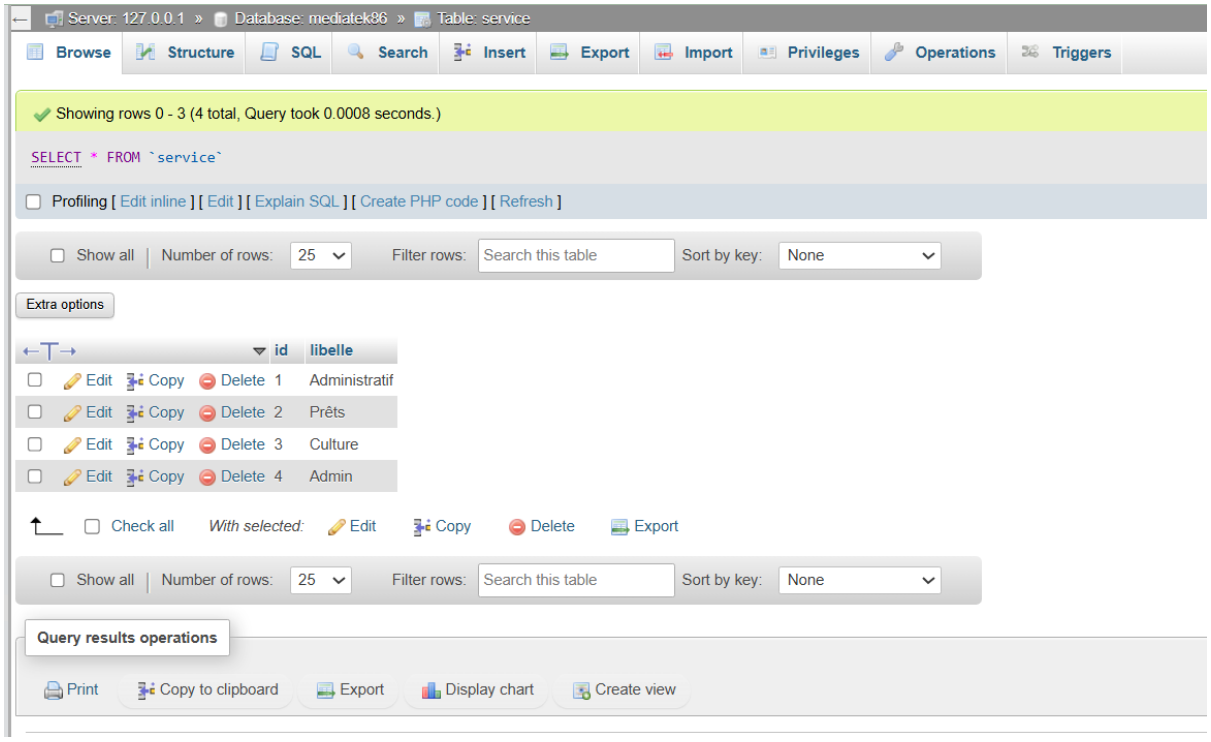
Concernant les droits d'accès :

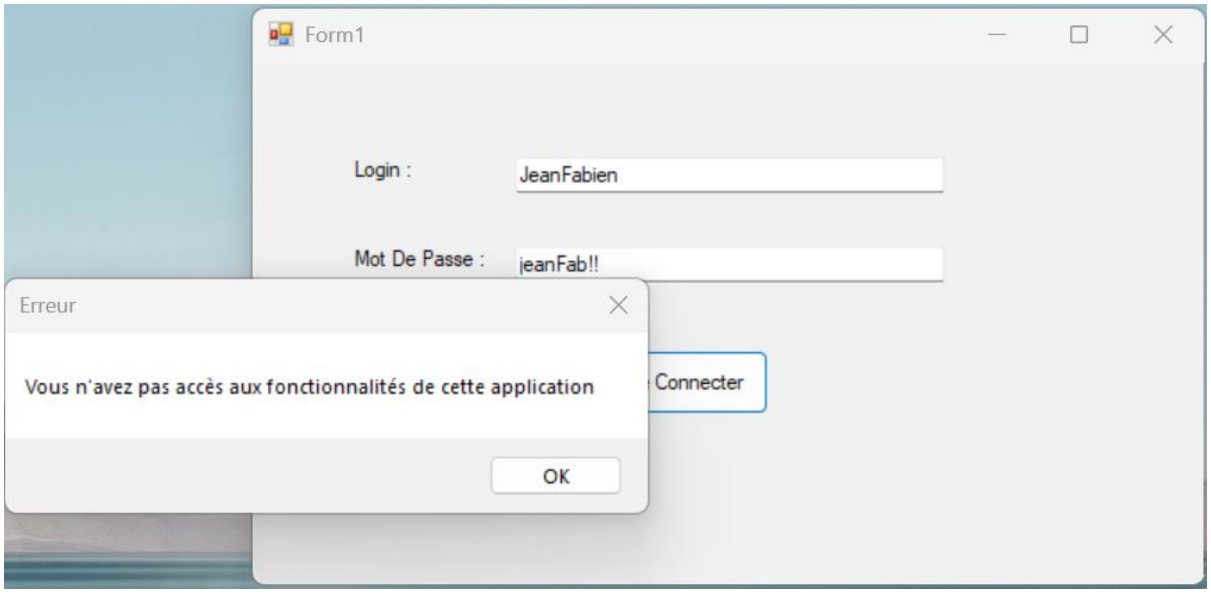
- Les utilisateurs du service "Prêts" ont un accès restreint ;
- Les boutons d'ajout, modification et suppression sont masqués ;
- Certains onglets (commandes, réception) sont désactivés ;
- Les autres utilisateurs disposent d'un accès complet.

Étapes suivies

Le développement a été réalisé en plusieurs étapes :

- Création du formulaire de connexion (loginPage) ;
- Appel à la méthode `GetUtilisateur` via le contrôleur ;
- Vérification du profil utilisateur ;
- Ouverture de la fenêtre principale avec les informations de l'utilisateur ;
- Mise en place de la méthode `GererDroits` ;
- Adaptation de l'interface selon `IdService`.





Résultat obtenu

L'accès à l'application est sécurisé grâce à un système d'authentification fonctionnel.
L'interface s'adapte automatiquement selon le profil utilisateur, garantissant que chaque utilisateur n'accède qu'aux fonctionnalités qui lui sont autorisées.

```
78
79          private readonly Utilisateur utilisateurConnecte;
80
81          /// <summary>
82          /// Constructeur : création du contrôleur lié à ce formulaire
83          /// </summary>
            1 reference | webby!, 12 days ago | 1 author, 1 change
84          internal FrmMediatek(Utilisateur utilisateur)
85          {
86              InitializeComponent();
87              this.controller = new FrmMediatekController();
88              this.utilisateurConnecte = utilisateur;
89          }
90
            1 reference | webby!, 11 days ago | 1 author, 2 changes
91          private void GererDroits()
92          {
93
94              if (utilisateurConnecte.IdService == "2")
95              {
96                  AppliquerDroitsPrets();
97              }
98
99
100         }
```

Mission 5 – Assurer la sécurité, la qualité et intégrer des logs

Tâche 1 – Corriger des problèmes de sécurité

Demande de la tâche

Cette tâche consistait à corriger des problèmes de sécurité liés à l'accès à l'API.
Elle impliquait également l'utilisation de GitHub (issues, branches, pull requests) pour organiser et valider les corrections.

Temps estimé : 4 heures.
Temps réel : 6 heures.

Travail réalisé

Deux problèmes de sécurité ont été corrigés :

- Les identifiants d'accès à l'API (login:pwd), initialement présents en dur dans le code, ont été déplacés dans le fichier `App.config` afin de sécuriser ces informations ;
- L'accès direct à l'API via le navigateur sans paramètres a été bloqué grâce à une configuration du fichier `.htaccess`, qui renvoie désormais une erreur.

L'organisation du travail a été réalisée avec GitHub :

- Création d'issues pour chaque problème ;
- Création de branches dédiées ;
- Mise en place de pull requests pour valider les corrections.

Étapes suivies

- Identification des problèmes de sécurité ;
- Création des issues dans GitHub ;
- Développement des corrections sur des branches dédiées ;
- Mise en place des pull requests ;
- Validation des corrections après vérification.

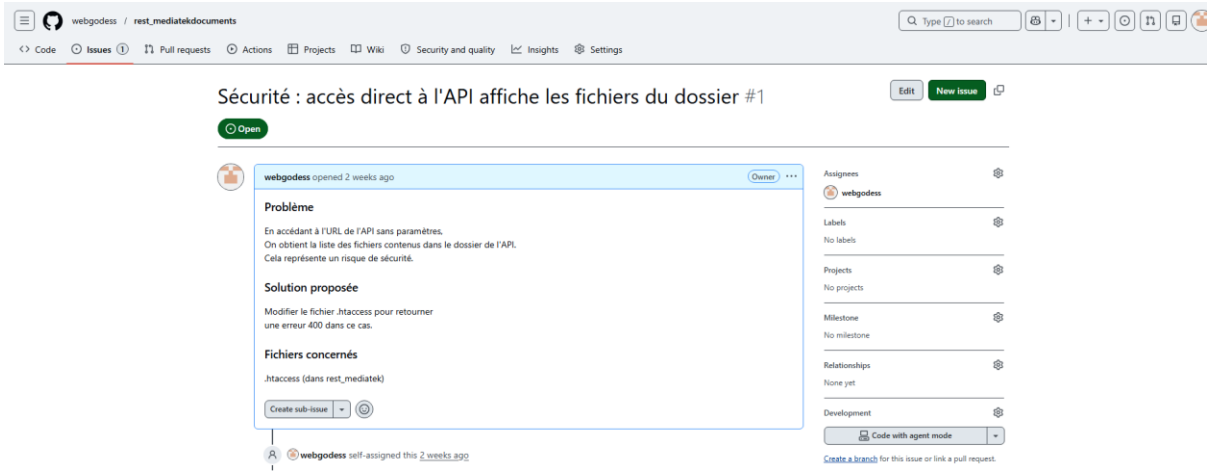
Résultat obtenu

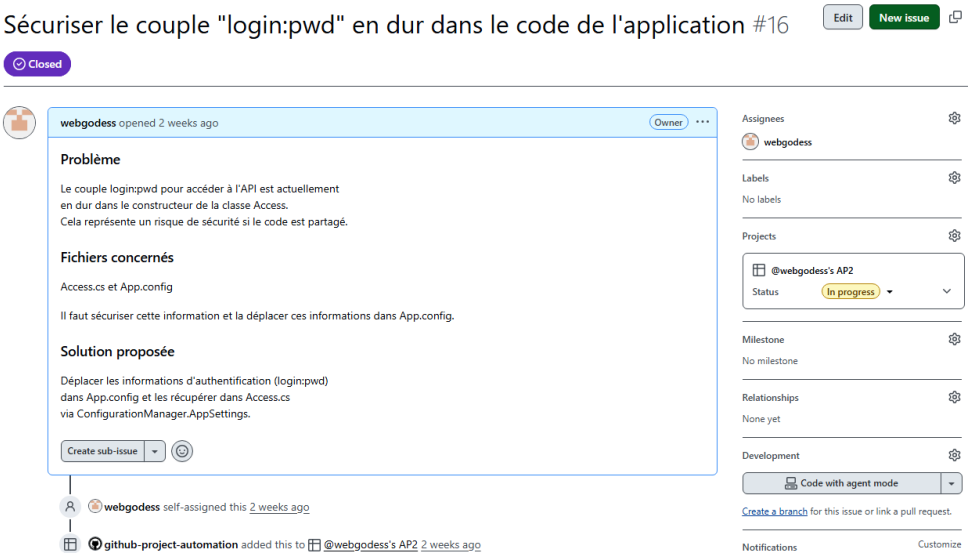
Les problèmes de sécurité ont été corrigés. Les informations sensibles ne sont plus exposées dans le code et l'accès à l'API est désormais protégé.

```
    <appSettings>
        <add key="ApiUrl" value="http://localhost/rest_mediatekdocuments/" />
        <add key="ApiLogin" value="admin" />
        <add key="ApiPwd" value="adminpwd" />
    </appSettings>
</configuration>
```

C: > xampp > htdocs > rest_mediatekdocuments > .htaccess

```
RewriteEngine on
RewriteCond %{HTTP:Authorization} ^(.+)$
RewriteRule ^(.*)$ - [E=HTTP_AUTHORIZATION:%1]
# Si la requete n'a pas de parametres, rediriger avec une erreur 400
RewriteRule ^$ - [R=400,L]
RewriteCond %{REQUEST_METHOD} =GET
RewriteRule ^([a-zA-Z0-9_]+)$ src/index.php?table=$1 [B,L]
RewriteCond %{REQUEST_METHOD} =GET
RewriteRule ^([a-zA-Z0-9_]+)/({.*})$ src/index.php?table=$1&champs=$2 [B,L]
RewriteCond %{REQUEST_METHOD} =POST
RewriteRule ^([a-zA-Z0-9_]+)$ src/index.php?table=$1 [B,L]
RewriteCond %{REQUEST_METHOD} =PUT
RewriteRule ^([a-zA-Z0-9_]+)$ src/index.php?table=$1 [B,L]
RewriteCond %{REQUEST_METHOD} =PUT
RewriteRule ^([a-zA-Z0-9_]+)/([a-zA-Z0-9]+)$ src/index.php?table=$1&id=$2 [B,L]
RewriteCond %{REQUEST_METHOD} =DELETE
RewriteRule ^([a-zA-Z0-9_]+)$ src/index.php?table=$1 [B,L]
RewriteCond %{REQUEST_METHOD} =DELETE
RewriteRule ^([a-zA-Z0-9_]+)/({.*})$ src/index.php?table=$1&champs=$2 [B,L]
```





Tâche 2 – Contrôler la qualité

Demande de la tâche

Cette tâche consistait à nettoyer le code ajouté en respectant les recommandations de SonarLint / SonarQube et les bonnes pratiques de développement.

Travail réalisé

Le code ajouté a été nettoyé afin d'améliorer sa qualité et sa maintenabilité.

Les principales améliorations sont :

- Correction des avertissements remontés par SonarLint ;
- Amélioration de la lisibilité du code ;
- Respect des conventions de nommage ;
- Structuration du code avec des constantes et des conditions plus claires.

Les méthodes événementielles n'ont pas été modifiées afin de respecter les contraintes liées aux composants graphiques.

Étapes suivies

- Analyse des alertes SonarLint ;
- Correction progressive du code ;
- Vérification du bon fonctionnement après modifications.

Résultat obtenu

Le code est plus propre, plus lisible et conforme aux standards de développement, sans impact sur le fonctionnement de l'application.

Tâche 3 – Intégrer des logs

Demande de la tâche

Cette tâche consistait à mettre en place un système de logs afin d'enregistrer les informations de fonctionnement de l'application.

Travail réalisé

Un système de logs a été intégré dans la classe `Access` à l'aide de la bibliothèque Serilog.

Les actions réalisées sont :

- Configuration du système de logs ;
- Enregistrement des messages dans un fichier de logs ;
- Remplacement des affichages `Console.WriteLine` par des logs.

Les logs sont enregistrés dans un fichier avec une rotation quotidienne, ce qui permet de conserver un historique des événements.

Étapes suivies

- Mise en place de la bibliothèque Serilog dans la classe Access.
- Définition du fichier de sortie des logs ;
- Remplacement des affichages console par des appels à `Log.Information` et `Log.Error` ;

```
95          String authenticationString;
96          // point de départ du log
97          Log.Logger = new LoggerConfiguration()
98              .MinimumLevel.Debug()
99              .WriteTo.Console()
100             .WriteTo.File("logs/log.txt", rollingInterval: RollingInterval.Day)
101             .CreateLogger();
102
103         try
104         {
105             // pour recuperer les clés du App.config
106             string apiUrl = ConfigurationManager.AppSettings["ApiUrl"];
107             string login = ConfigurationManager.AppSettings["ApiLogin"];
108             string pwd = ConfigurationManager.AppSettings["ApiPwd"];
109
110             // on remplace dans l'authentification string
111             authenticationString = $"{login}:{pwd}";
112
113             api = ApiRest.GetInstance(apiUrl, authenticationString);
114             Log.Information("Accès API initialisé avec succès. URL API : {ApiUrl}"
115         }
116         catch (Exception e)
117         {
118
119             Log.Error(e, "Erreur fatale lors de l'initialisation de l'accès API");
120             Environment.Exit(0);
121         }
122     }
```

```
2026-04-12 13:08:35.989 +02:00 [INF] Appel API - Méthode : PUT, Ressource : commandedocument/00045, Paramètres :
champs={"idSuivi":"10003"}
2026-04-12 13:08:36.018 +02:00 [INF] Appel API - Méthode : GET, Ressource :
commandedocument/{"idLivreDvd":"20004"}, Paramètres : null
2026-04-12 13:08:36.055 +02:00 [INF] Appel API - Méthode : GET, Ressource : commande, Paramètres : null
2026-04-12 13:09:10.365 +02:00 [INF] Appel API - Méthode : DELETE, Ressource : commande/{"id":"00045"},
Paramètres : null
2026-04-12 13:09:10.452 +02:00 [INF] Appel API - Méthode : GET, Ressource :
commandedocument/{"idLivreDvd":"20004"}, Paramètres : null
2026-04-12 13:09:10.497 +02:00 [INF] Appel API - Méthode : GET, Ressource : commande, Paramètres : null
2026-04-12 15:05:11.160 +02:00 [INF] Accès API initialisé avec succès. URL API :
http://localhost/rest_mediatekdocuments/
2026-04-12 15:05:31.070 +02:00 [INF] Appel API - Méthode : GET, Ressource : utilisateur/{"login":"admin"},
Paramètres : null
2026-04-12 15:23:04.240 +02:00 [INF] Accès API initialisé avec succès. URL API :
http://localhost/rest_mediatekdocuments/
2026-04-12 15:23:47.531 +02:00 [INF] Appel API - Méthode : GET, Ressource : utilisateur/{"login":"Pierrick"},
Paramètres : null
2026-04-12 15:23:48.243 +02:00 [INF] Appel API - Méthode : GET, Ressource : abonnement, Paramètres : null
```

Résultat obtenu

Les logs sont correctement enregistrés dans un fichier, ce qui permet de suivre l'activité de l'application et de faciliter le diagnostic des erreurs.

Mission 6 – Tester et documenter

Temps estimé : 8 heures
Temps réel : 10 heures

Tâche 1 – Gérer les tests

Demande de la tâche

Cette tâche consistait à mettre en place des tests afin de vérifier le bon fonctionnement de l'application et de l'API.
Elle comprenait la réalisation de tests unitaires sur les classes du package Model ainsi que des tests fonctionnels via Postman.

Travail réalisé

Des tests ont été mis en place pour valider le bon fonctionnement des différentes fonctionnalités.

Concernant les tests unitaires :

- Des tests ont été écrits sur les classes du package Model ;

Concernant les tests fonctionnels :

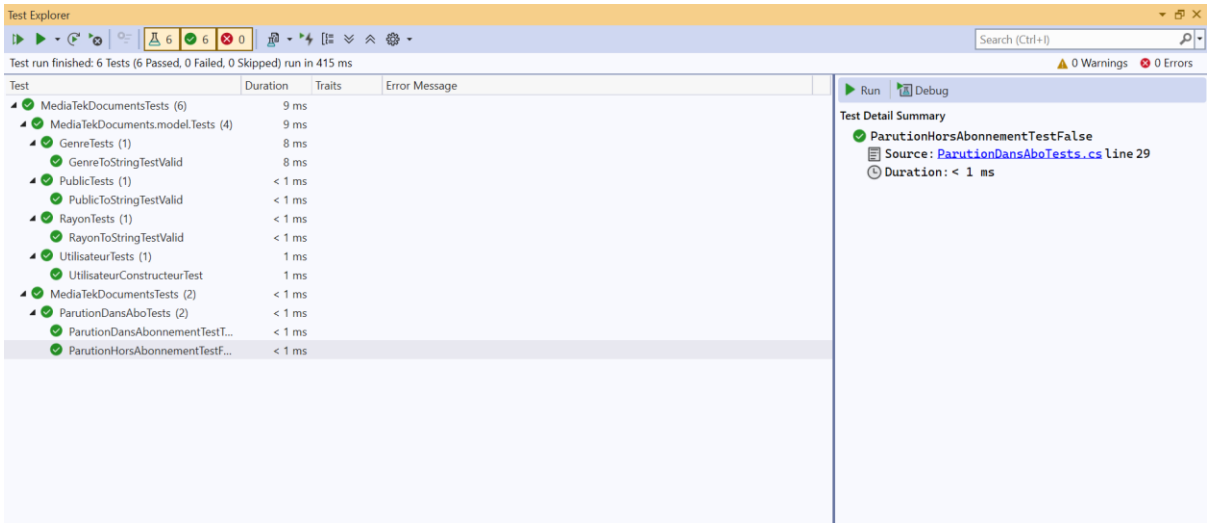
- Une collection de tests a été créée dans Postman ;
- Les principales routes de l'API ont été testées (GET, POST, PUT, DELETE) ;
- Les réponses de l'API ont été vérifiées afin de s'assurer de leur conformité.

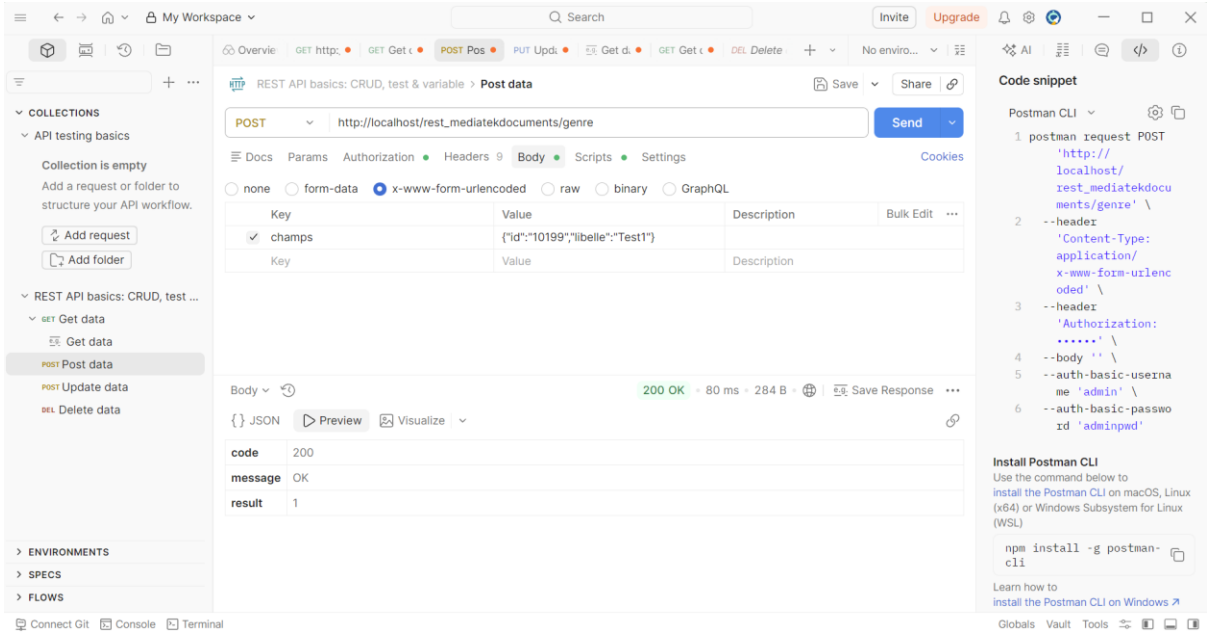
Étapes suivies

- Création du projet de tests unitaires ;
- Écriture des tests sur les classes du Model ;
- Création de tests Postman ;

Résultat obtenu

Les tests unitaires et fonctionnels permettent de valider le bon fonctionnement de l'application et de l'API.
Ils assurent une meilleure fiabilité du système et facilitent la détection d'éventuelles erreurs.





Tâche 2 – Créer les documentations techniques

Demande de la tâche

Cette tâche consistait à générer les documentations techniques de l'application C# et de l'API REST à partir des commentaires du code.

Travail réalisé

Les documentations techniques ont été générées pour l'application C# et pour l'API REST.

Les actions réalisées sont :

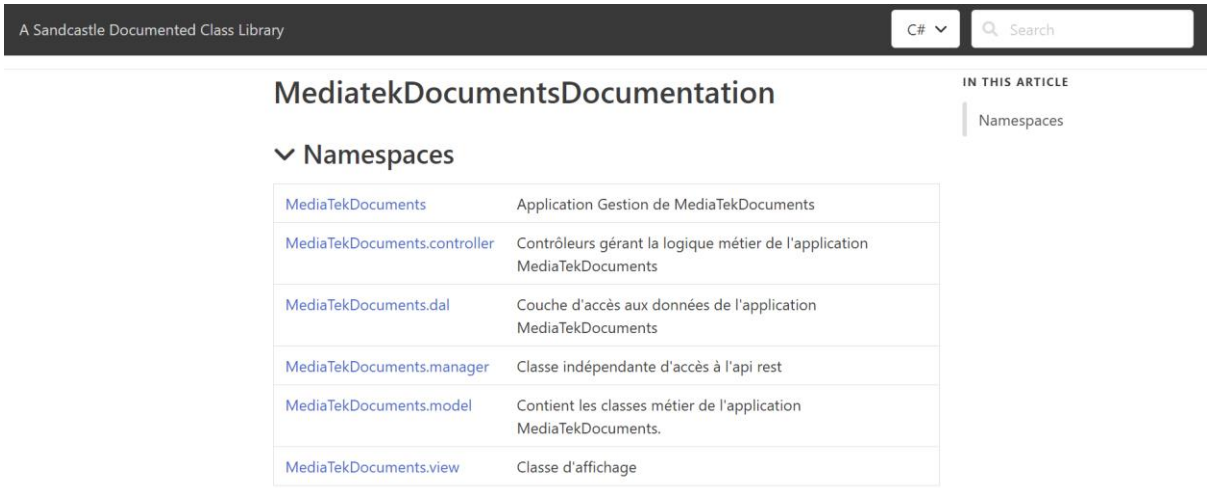
- Vérification et ajout de commentaires normalisés dans le code ;
- Génération de la documentation technique de l'application C# ;
- Génération de la documentation de l'API REST ;
- Transfert des documentations dans les dépôts.

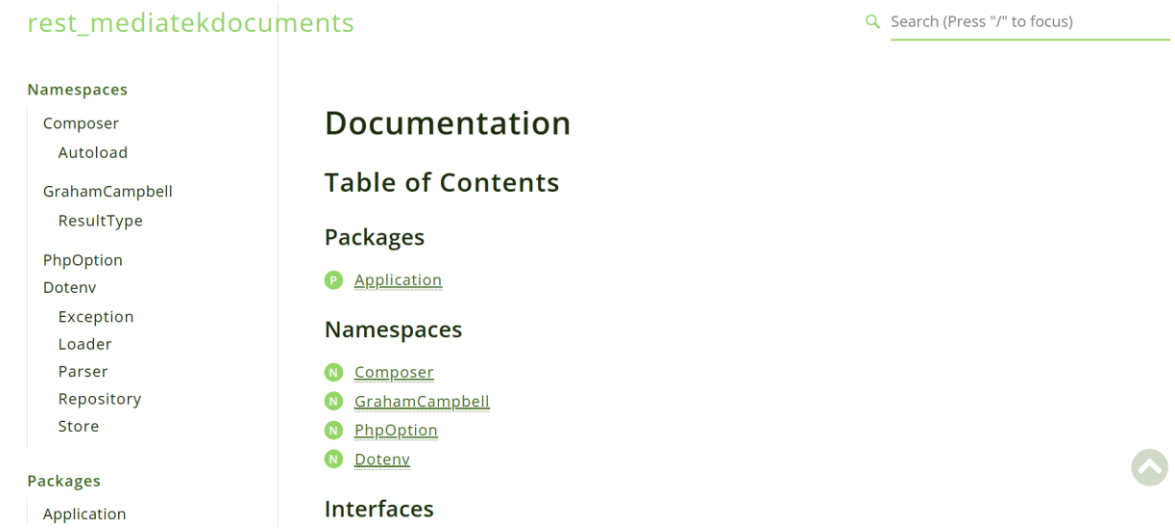
Étapes suivies

- Vérification des commentaires dans les classes et méthodes ;
- Ajout des commentaires manquants ;
- Génération des documentations via les outils adaptés ;
- Dépôt des fichiers générés.

Résultat obtenu

Les documentations techniques sont complètes et permettent de comprendre la structure et le fonctionnement de l'application ainsi que de l'API.





Tâche 3 – Créer la documentation utilisateur en vidéo

Demande de la tâche

Cette tâche consistait à réaliser une vidéo de présentation de l'application C#, d'une durée maximale de 10 minutes.

Travail réalisé

Une vidéo de démonstration a été réalisée afin de présenter les principales fonctionnalités de l'application.

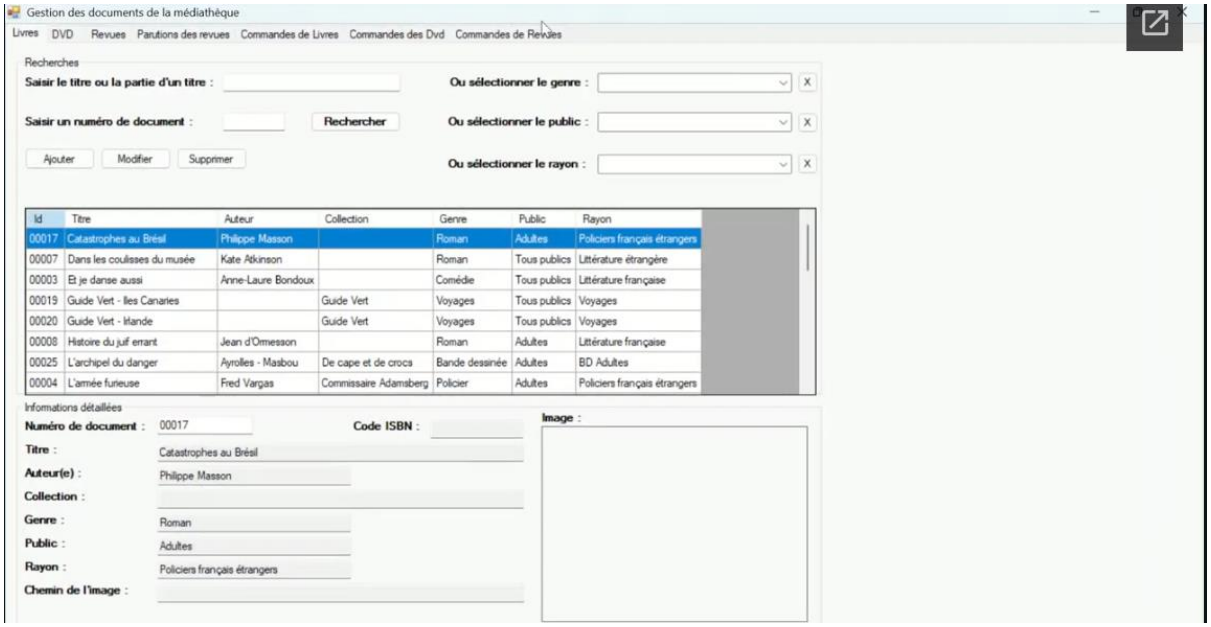
La vidéo montre notamment :

- La connexion utilisateur ;
- La gestion des livres, DVD et revues ;
- La gestion des commandes ;
- Les fonctionnalités principales de l'application.

Un outil de capture vidéo a été utilisé pour enregistrer la démonstration.

Étapes suivies

- Enregistrement de la vidéo ;



Résultat obtenu

La vidéo permet de présenter clairement l'application et ses fonctionnalités.
Elle constitue une documentation utilisateur accessible et facile à comprendre.

Conclusion générale

Ce projet a permis de développer une application de gestion de médiathèque en C# avec une API REST, en mobilisant des compétences en développement, base de données, sécurité, tests et documentation.

Les différentes missions ont permis de mettre en place des fonctionnalités complètes (commandes, abonnements, authentification, logs) tout en respectant les bonnes pratiques et en utilisant des outils professionnels comme GitHub, Postman et SonarLint.

Je n'ai pas pu mettre en place la mission 7 dans le temps imparti.